UNIVERSIDAD DE ORIENTE (UNIVO)

Facultad: Ingeniería y Arquitectura

Carrera: Ingeniería en Desarrollo de Software

Gestión de Servidores Web

Laboratorio 2

Servicio Web Apache, Certificados SSL y Proxy Inverso

Caso de estudio: “Dulce Hogar”

Asignatura	Gestión de Servidores Web
Laboratorio	Laboratorio 2 — Unidad 2
Docente	Turcios Ortez Alberto Manases
Equipo	Equipo 04
Fecha de entrega	11 de octubre de 2026

Integrantes del Equipo 04

N.º	Nombre completo	Rol asignado
1	Carlos Josue Majano Granados	Coordinador / Git
2	Dereck Enrique Benitez Ventura	Adm. de Servidor Base
3	Luis Roman Lino Merino	Adm. de Redes / Proxy
4	Jose Santos Aguilar	Adm. de Certificados (SSL)
5	Andersson Wilfredo Vargas Reyes	Adm. de Permisos (OS)
6	Edwin David Díaz Acosta	Documentador / QA

Contenido

1. Introducción y objetivo
2. Integrantes y distribución de roles
3. Fase 1 — Dimensionamiento colaborativo
4. Fase 2 — Instalación y verificación de Apache
5. Fase 3 — VirtualHost y Reverse Proxy
6. Fase 4 — HTTPS local con mkcert
7. Evidencias (capturas)
8. Estructura del repositorio
9. Conclusiones

1. Introducción y objetivo

El presente informe documenta el desarrollo del **Laboratorio 2 de la Unidad 2** de la asignatura **Gestión de Servidores Web**. El trabajo se realizó de forma colaborativa tomando como caso de estudio la pequeña empresa **“Dulce Hogar”**, que requiere publicar un sitio web local con su API, bajo un esquema seguro y profesional.

El laboratorio se dividió en cuatro fases: (1) dimensionamiento colaborativo del servidor, (2) instalación y verificación de Apache, (3) configuración de VirtualHost y Reverse Proxy hacia la API, y (4) habilitación de HTTPS local mediante certificados de `mkcert`. Cada fase fue responsabilidad de un integrante del equipo, con revisión cruzada del resto.

Objetivo general:

Desplegar y documentar un servidor web Apache funcional para el dominio local `dulcehogar.local`, que sirva contenido estático, actúe como proxy inverso hacia una API en el puerto 9000 y exponga el sitio por HTTPS en el puerto 8900 sin advertencias de seguridad.

Entorno utilizado:

- Windows con **WSL2** y distribución **Ubuntu Server LTS**.
- **Apache 2.4.66-2ubuntu2.5** con `systemd` activo.
- API de pruebas escuchando en el puerto interno **9000**.
- **mkcert v1.4.4** para la Autoridad Certificadora local.

2. Integrantes y distribución de roles

El equipo 04 organizó el trabajo por roles especializados, garantizando que cada fase tuviera un responsable claro y un revisor.

Integrante	Rol	Responsabilidad principal	Fases
Carlos Josue Majano Granados	Coordinador / Git	Gestión del repositorio, organización y consolidación	Fase 1, Estructura, README
Dereck Enrique Benitez Ventura	Adm. de Servidor Base	Instalación y verificación de Apache	Fase 2
Luis Roman Lino Merino	Adm. de Redes / Proxy	Configuración de VirtualHosts y Reverse Proxy	Fase 3
Jose Santos Aguilar	Adm. de Certificados (SSL)	Instalación de mkcert y SSL en el puerto 8900	Fase 4
Andersson Wilfredo Vargas Reyes	Adm. de Permisos (OS)	Menor privilegio (www-data) y CA en Windows	Fase 1, Fase 4
Edwin David Diaz Acosta	Documentador / QA	Redacción del informe PDF y pruebas de integración	Documentación y QA

3. Fase 1 — Dimensionamiento colaborativo

Responsables: Ander (Adm. de Permisos / OS) y Carlos Majano (Coordinación). A continuación se reproduce el análisis del caso **Dulce Hogar**.

Datos del caso

- Sistema operativo: Ubuntu Server LTS (con WSL2 en máquinas Windows).
- Hardware: 2 vCPU, 4 GB de RAM y 40 GB SSD.
- Tráfico normal: 500 visitas diarias.
- Tráfico máximo estimado: 3,000 visitas en días festivos.

1. ¿Los 2 vCPU y 4 GB de RAM son suficientes? ¿Cuál es el factor pico?

Los 2 vCPU y los 4 GB de RAM pueden ser suficientes para atender las 500 visitas diarias, siempre que el sitio esté bien optimizado. Cuando el tráfico llegue a 3,000 visitas en días festivos, podrían presentarse problemas de rendimiento dependiendo de cuántas personas entren al mismo tiempo. El factor pico es **6**, porque 3,000 dividido entre 500 es igual a 6.

```
Factor pico = 3,000 / 500 = 6 veces.
```

2. ¿Escalamiento vertical u horizontal?

Recomendamos el **escalamiento vertical** para Dulce Hogar, ya que es una pequeña empresa y mejorar el servidor actual puede ser más sencillo y económico al inicio. Si el crecimiento del negocio hace que un solo servidor ya no sea suficiente, se podría considerar el **escalamiento horizontal** para distribuir las solicitudes entre varios servidores, aunque esto requiere mayor configuración y mantenimiento.

3. ¿Qué recurso vigilar primero si el servidor está lento?

Primero revisaríamos el uso de la **CPU** y la **memoria RAM**, porque son recursos importantes para procesar las solicitudes y mantener los servicios funcionando. También comprobaríamos el espacio y la actividad del disco, además del tráfico de red, para identificar cuál de los cuatro recursos está causando la lentitud antes de aumentar la capacidad del servidor.

- CPU: revisar si permanece cerca del 100% durante un periodo prolongado.
- RAM: comprobar la memoria disponible y el uso de swap.
- Disco: revisar el espacio libre y la actividad de lectura/escritura.
- Red: comprobar si existe saturación o problemas de conexión.

4. Decisión del equipo: ¿nube o servidor local?

Elegimos la **nube** para Dulce Hogar porque permite comenzar con recursos modestos y ampliarlos cuando aumenten las visitas. Además, facilita el mantenimiento de la infraestructura física y evita una inversión inicial elevada en equipos propios. Aunque el servidor local ofrece mayor control directo, para una pequeña empresa la nube puede ser una alternativa más flexible.

Criterio	Nube	Servidor local
Costo inicial	Generalmente bajo, no requiere comprar un servidor físico propio.	Puede ser alto por la compra del equipo y sus componentes.
Escalabilidad	Es más sencillo aumentar recursos cuando crece el tráfico.	Depende del hardware disponible y de las ampliaciones posibles.
Mantenimiento	El proveedor mantiene parte de la infraestructura física.	La empresa se encarga del equipo y su mantenimiento.
Control	Depende de los servicios y permisos del proveedor.	Mayor control directo sobre el equipo y su configuración.

4. Fase 2 — Instalación y verificación de Apache

Responsable de la fase: Administrador de servidor base (Apoyo: Coordinador). Ejecutado por: **Dereck Ventura**.

Entorno: Windows con WSL2 y Ubuntu (repositorios “resolute”), Apache **2.4.66-2ubuntu2.5** y systemd activo en WSL2 (se usó `systemctl` sin problemas).

Comandos ejecutados y salidas

```
LABORATORIO 2 - FASE 2: Instalación de Apache
Caso: "Dulce Hogar" - dulcehogar.local

Responsable de la fase: Administrador de servidor base (Apoyo: Coordinador)
Ejecutado por: Dereck Ventura

Entorno utilizado:
- Windows con WSL2 y Ubuntu (repositorios "resolute")
- Apache 2.4.66-2ubuntu2.5
- systemd activo en WSL2 (se usó systemctl sin problemas)

1. COMANDOS EJECUTADOS Y SALIDAS (entorno propio del equipo)

$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
[... listas de paquetes de main, universe, restricted y multiverse ...]
Fetched 32.1 MB in 12s (2732 kB/s)
103 packages can be upgraded. Run 'apt list --upgradable' to see them.

$ sudo apt install -y apache2
Installing:
  apache2
Installing dependencies:
  apache2-bin  apache2-data  apache2-utils  libapr1t64  libaprutil1-dbd-sqlite3
  libaprutil1-ldap  libaprutil1t64  liblua5.4-0  ssl-cert
Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 103
  Download size: 2120 kB
  Space needed: 8218 kB / 1024 GB available
Fetched 2120 kB in 2s (893 kB/s)
[... descarga y desempaquetado de los 10 paquetes ...]
Setting up apache2 (2.4.66-2ubuntu2.5) ...
Enabling module mpm_event.
Enabling module authz_core.
Enabling module authz_host.
Enabling module alias.
Enabling module dir.
Enabling module deflate.
Enabling module status.
```

```
[... otros módulos y configuraciones básicas ...]
Enabling site 000-default.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apa
Created symlink '/etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service' → '/usr/lib/syste

$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-11 17:00:38 CST; 29min ago
 Invocation: b2aa4f6cef414e4bac91e61005e72b2a
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1497 (apache2)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 55 (limit: 2155)
     Memory: 5.4M (peak: 5.8M)
        CPU: 930ms
     CGroup: /system.slice/apache2.service
             ■■1497 /usr/sbin/apache2 -k start -DFOREGROUND
             ■■1513 /usr/sbin/apache2 -k start -DFOREGROUND
             ■■1515 /usr/sbin/apache2 -k start -DFOREGROUND
Oct 11 17:00:37 LAPTOP-GGOC9T3P systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 11 17:00:38 LAPTOP-GGOC9T3P systemd[1]: Started apache2.service - The Apache HTTP Server.

$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

$ curl http://localhost
(Salida recortada: se muestran las líneas más relevantes del HTML devuelto)
&lt;!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" ...&gt;
&lt;html xmlns="http://www.w3.org/1999/xhtml"&gt;
  &lt;head&gt;
    &lt;title&gt;Apache2 Ubuntu Default Page: It works&lt;/title&gt;
    [... estilos CSS ...]
  &lt;/head&gt;
  &lt;body&gt;
    [...]
          Apache2 Default Page
          It works!
    [...]
    This is the default welcome page used to test the correct operation of
    the Apache2 server after installation on Ubuntu systems. [...] You should
    replace this file (located at /var/www/html/index.html) before continuing
    to operate your HTTP server.
    [... sección "Configuration Overview", "Document Roots" y "Reporting Problems" ...]
  &lt;/body&gt;
&lt;/html&gt;

$ ls /etc/apache2/sites-available/
000-default.conf  default-ssl.conf

$ ls /var/www/html/
index.html

$ sudo apache2ctl configtest
Syntax OK
```

Preguntas para discutir

Pregunta 1. ¿Qué muestra la página por defecto de Apache?

Muestra la página “Apache2 Ubuntu Default Page: It works!”. Confirma que Apache se instaló correctamente y está atendiendo peticiones HTTP en el puerto 80. La página explica la estructura

de configuración de Apache en Ubuntu (apache2.conf, ports.conf y los directorios mods-enabled, conf-enabled y sites-enabled) y que el DocumentRoot por defecto es /var/www/html, donde está el index.html que debe reemplazarse antes de poner el sitio en producción. Evidencia: fase2-apache-activo.png.

Pregunta 2. ¿Qué diferencia hay entre sites-available y sites-enabled?

/etc/apache2/sites-available/ guarda todos los archivos de configuración de Virtual Hosts disponibles, estén activos o no (000-default.conf y default-ssl.conf). /etc/apache2/sites-enabled/ contiene únicamente enlaces simbólicos a los sitios activos; Apache solo lee esta carpeta al arrancar. Los comandos a2ensite crea el enlace (activa el sitio) y a2dissite lo elimina (desactiva) sin borrar el archivo original.

Pregunta 3. Ejecuten sudo apache2ctl configtest. ¿Qué reporta? ¿Por qué es importante?

Reporta “Syntax OK”, lo que significa que la sintaxis de todos los archivos de configuración es válida. Es importante ejecutarlo antes de reiniciar o recargar porque, si hubiera un error y se reiniciara directamente, Apache no arrancaría y el sitio quedaría caído. Con configtest el error se detecta mientras el servicio anterior sigue funcionando y se puede corregir sin interrumpir el servicio.

Pregunta 4 (Reto). Si Apache no arrancara, ¿qué 3 pasos seguirían para diagnosticar?

1) Revisar el estado del servicio y su mensaje de error (systemctl status apache2 / journalctl -xeu apache2). 2) Validar la configuración y revisar los registros (apache2ctl configtest / tail -n 50 /var/log/apache2/error.log). 3) Verificar conflictos de puerto y módulos (ss -tlnp | grep :80; si otro servicio como IIS usa el puerto 80 se detiene o se cambia en ports.conf; también se confirman los módulos con apache2ctl -M).

Errores encontrados y cómo se resolvieron

Error 1 — WslRegisterDistribution failed with error: 0x800701bc

- Mensaje: “WSL 2 requiere una actualización de su componente de kernel.”
- Causa: el kernel de WSL2 en Windows estaba desactualizado.
- Solución: se actualizó WSL desde PowerShell como administrador con `wsl --update` y luego `wsl --shutdown`.

Error 2 — WslRegisterDistribution failed with error: 0x80370114

- Mensaje: “No se pudo iniciar la operación porque no se instaló una característica requerida.”
- Causa: faltaba una característica de Windows necesaria para WSL2 o la virtualización no estaba disponible/activada.
- Solución: activar las características “Subsistema de Windows para Linux” y “Plataforma de máquina virtual” con dism, reiniciar la PC y/o activar la virtualización en la BIOS.

Resultado: Ubuntu se instaló y funciona correctamente en WSL2.

Evidencia y entregables de la fase

- Captura: capturas/fase2-apache-activo.png.
- Apache instalado, activo (running) y habilitado en el arranque.
- Configuración validada con apache2ctl configtest: Syntax OK.

5. Fase 3 — VirtualHost y Reverse Proxy

Integrante responsable: **Luis Roman** (Adm. de Redes / Proxy).

1. ¿Qué hacen exactamente ProxyPass y ProxyPassReverse?

`ProxyPass` redirige el tráfico entrante que llega a la ruta `/api` del servidor Apache (puerto 80) hacia la aplicación backend que escucha en el puerto interno 9000. `ProxyPassReverse` reescribe los encabezados HTTP (como *Location* o *URI*) en la respuesta que devuelve el servidor backend de la API, garantizando que el cliente/navegador nunca vea la IP interna ni el puerto 9000, manteniendo la transparencia y seguridad del Reverse Proxy.

2. Si apache2ctl configtest falla, ¿cuáles son los 2 errores más comunes?

Los dos errores más comunes son: **a)** olvidar cerrar correctamente una etiqueta de bloque (por ejemplo, omitir `</VirtualHost>`); y **b)** errores tipográficos en las directivas (como escribir “Servername” o “ProxyPasss”) o falta de espacio entre los parámetros de origen y destino. Para depurarlos se ejecuta `sudo apache2ctl configtest` para que la terminal indique la línea exacta del error, o se revisan los últimos registros del sistema con `journalctl -xeu apache2`.

3. Demostración técnica de la conexión con la API

Se habilitaron los módulos `proxy` y `proxy_http`. Al ejecutar la prueba con cURL hacia la URL `http://dulcehogar.local/api/saludo?nombre=Equipo`, el VirtualHost procesa la petición en el puerto 80 y la redirige exitosamente al puerto 9000 de la API. Evidencia: capturas/fase3-curl-exitoso.png.

Configuración de los VirtualHost (puerto 80 y puerto 8900)

```
Listen 0.0.0.0:8900

<VirtualHost *:80>
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    <Location /api>
        Header set Content-Type "application/json"
    </Location>

    ProxyRequests Off
    ProxyPreserveHost On

    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api
</VirtualHost>

<VirtualHost *:8900>
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    SSLEngine on
    SSLCertificateFile /home/carlos/gsw-practico-unidad2-equipo04/certs/dulcehogar.local.pem
    SSLCertificateKeyFile /home/carlos/gsw-practico-unidad2-equipo04/certs/dulcehogar.local-key.pem

    <Location /api>
        Header set Content-Type "application/json"
    </Location>

    ProxyRequests Off
    ProxyPreserveHost On

    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api
</VirtualHost>
```

6. Fase 4 — HTTPS local con mkcert

Responsable: Administrador de Certificados (SSL), **Jose Aguilar**.

1) Comandos ejecutados (mkcert + SSL)

a) Instalación de mkcert

```
sudo apt update
sudo apt install -y libnss3-tools wget
wget -O mkcert https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-linux-amd64
chmod +x mkcert
sudo mv mkcert /usr/local/bin/mkcert
mkcert -version
```

b) Generación de la CA local y del certificado

```
mkcert -install
  (NOTA: el comando lleva GUION: mkcert -install. Sin guion crea un
   certificado basura para el dominio "install" y no instala la CA.)

mkcert dulcehogar.local
sudo mkdir -p /etc/apache2/ssl/dulcehogar.local
sudo mv dulcehogar.local.pem dulcehogar.local-key.pem /etc/apache2/ssl/dulcehogar.local/

Certificados generados:
  /etc/apache2/ssl/dulcehogar.local/dulcehogar.local.pem
  /etc/apache2/ssl/dulcehogar.local/dulcehogar.local-key.pem
```

c) Abrir el puerto 8900

```
echo "Listen 8900" | sudo tee -a /etc/apache2/ports.conf
```

d) Virtual Host HTTPS (config/dulcehogar.local-ssl.conf)

```
&lt;IfModule mod_ssl.c&gt;
  &lt;VirtualHost *:8900&gt;
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    SSLEngine on
    SSLCertificateFile   /etc/apache2/ssl/dulcehogar.local/dulcehogar.local.pem
    SSLCertificateKeyFile /etc/apache2/ssl/dulcehogar.local/dulcehogar.local-key.pem

    ProxyPreserveHost On
    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api
  &lt;/VirtualHost&gt;
&lt;/IfModule&gt;
```

e) Habilitar y verificar

```
sudo a2enmod ssl
sudo a2ensite dulcehogar.local-ssl.conf
sudo apache2ctl configtest      -&gt; Syntax OK
sudo systemctl restart apache2
sudo ss -tlnp | grep apache2    -&gt; *:8900 en escucha
```

f) Confianza de la CA en Windows

La CA raíz de mkcert se instaló en “Entidades de certificación raíz de confianza” de Windows (`rootCA.pem`). También se agregó en el archivo `hosts` de Windows: `172.20.201.178`

`dulcehogar.local`. En este equipo el puerto 80 lo ocupa IIS, por eso se publicó el sitio HTTPS en el puerto 8900, tal como pide la práctica.

2) ¿Por qué certbot no sirve para dulcehogar.local?

certbot usa Let's Encrypt, que solo emite certificados para dominios **públicos** reales. Para validar la propiedad del dominio, Let's Encrypt necesita que el servidor sea alcanzable desde internet (validación HTTP-01 o DNS-01). “dulcehogar.local” no es un dominio público: no existe en el DNS mundial y no es direccionable desde internet, por lo que la validación fallaría siempre. Por eso, para HTTPS en un entorno local se usa mkcert, que instala una CA raíz local y firma el certificado sin depender de internet.

3) Diferencia: certificado autofirmado vs certificado de mkcert

- **Autofirmado:** el propio servidor firma su certificado. Ningún navegador confía en esa firma, por lo que siempre aparece la advertencia “Su conexión no es privada”. Debe aceptarse manualmente cada vez.
- **mkcert:** crea una Autoridad Certificadora raíz **local** y la instala en el almacén de confianza del sistema (en WSL y en Windows). Como el navegador ya confía en esa CA, el certificado firmado por ella se valida sin advertencia (candado verde).
- Ninguno de los dos es válido para un dominio público real: para producción se necesita una CA pública (Let's Encrypt/comercial).

4) ¿Qué cambiaría en producción (dulcehogar.com.sv)?

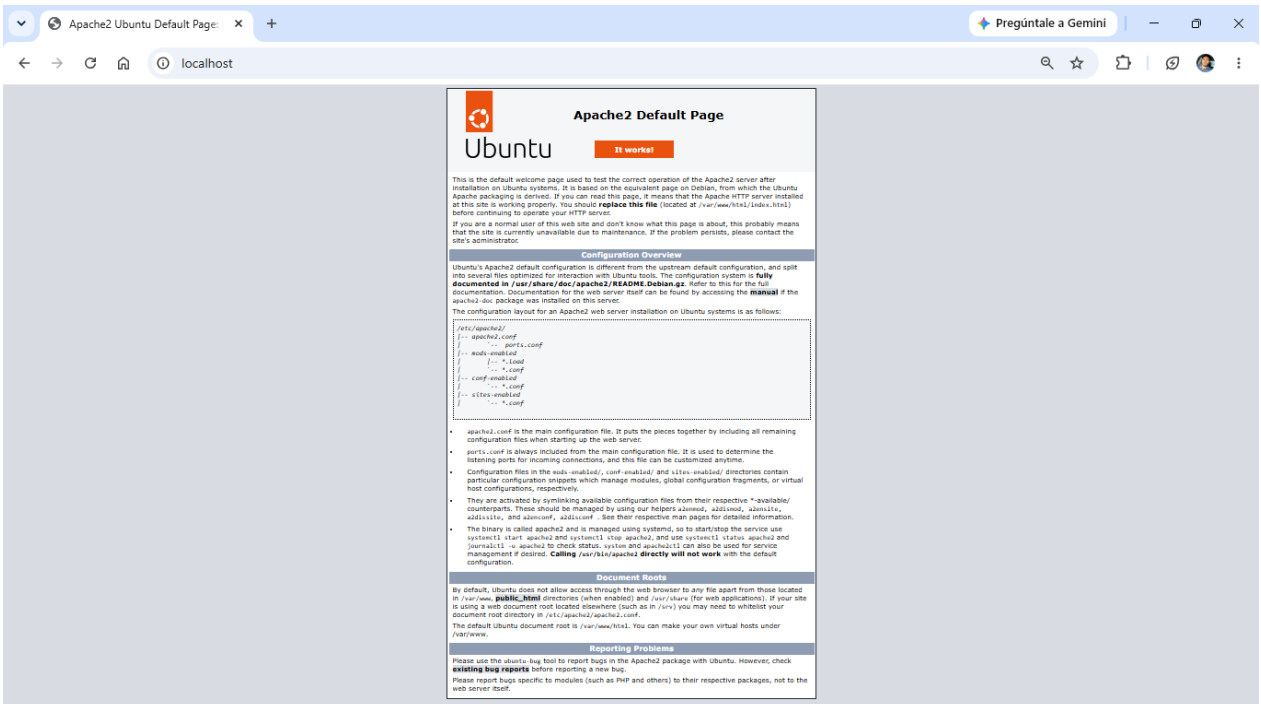
- Usar el dominio público real **dulcehogar.com.sv** con su DNS apuntando a una IP pública del servidor.
- Solicitar un certificado de una CA pública (p. ej. certbot/Let's Encrypt, con renovación automática) en lugar de mkcert.
- Escuchar en el puerto estándar **443** (https://dulcehogar.com.sv) y redirigir el puerto 80 a 443.
- Mantener el proxy a la API interna (localhost:9000) solo en la red privada y reforzar la seguridad (firewall, actualizaciones, HTTPS para la API si es posible).

5) Validación y capturas

Verificación de configuración: `sudo apache2ctl configtest` → **Syntax OK**. La página se abrió en el navegador en `https://dulcehogar.local:8900/` (y `https://dulcehogar.local:8900/api/saludo?nombre=Equipo` cuando la API esté disponible) **sin advertencia de seguridad**, con el candado de HTTPS.

7. Evidencias (capturas)

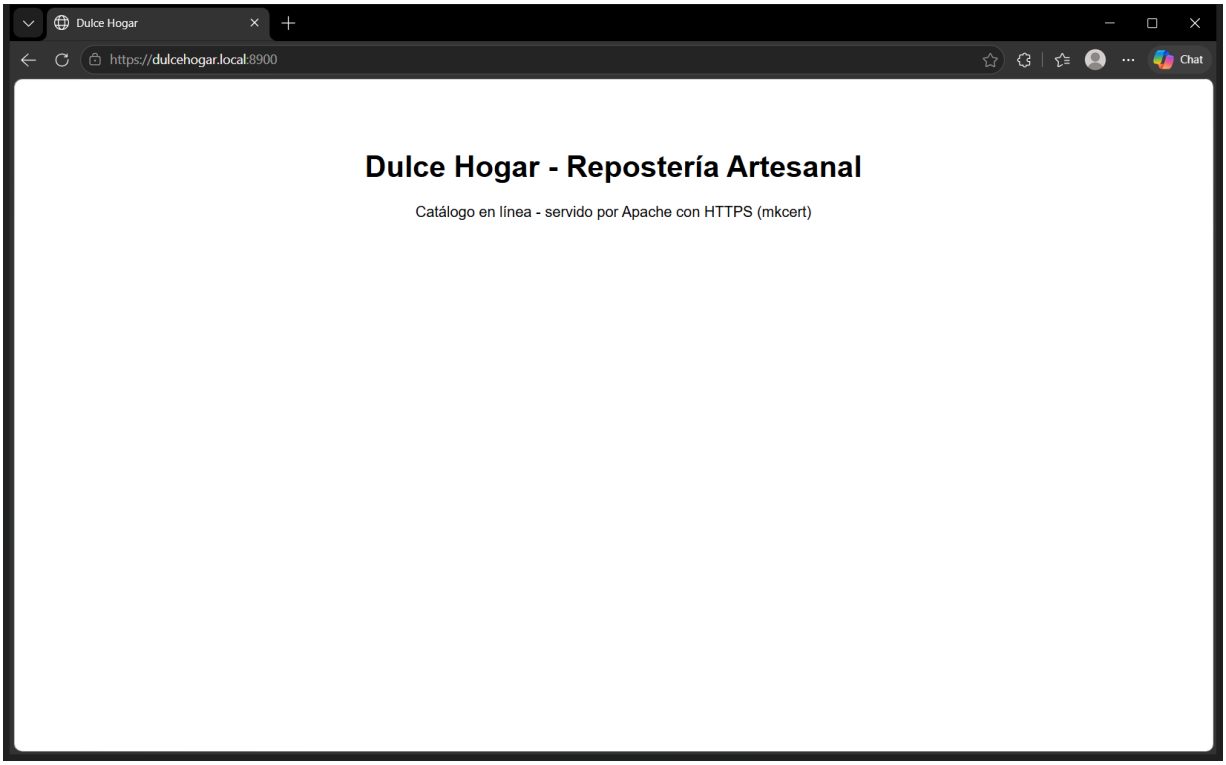
Las siguientes capturas respaldan las pruebas realizadas durante el laboratorio.



Fase 2 — Apache activo y atendiendo peticiones (página por defecto de Ubuntu).

```
luisr@ASUSROMAN:~$ sudo mkdir -p /var/www/dulcehogar.local/api
echo '{"mensaje": "Hola Equipo, bienvenido a la API de Dulce Hogar"}' | sudo tee /var/www/dulcehogar.local/api/saludo
{"mensaje": "Hola Equipo, bienvenido a la API de Dulce Hogar"}
luisr@ASUSROMAN:~$ curl http://localhost:9000/api/saludo?nombre=Equipo
127.0.0.1 - - [11/Oct/2026 12:38:26] "GET /api/saludo?nombre=Equipo HTTP/1.1" 200 -
{"mensaje": "Hola Equipo, bienvenido a la API de Dulce Hogar"}
luisr@ASUSROMAN:~$ curl http://dulcehogar.local/api/saludo?nombre=Equipo
127.0.0.1 - - [11/Oct/2026 12:38:49] "GET /api/saludo?nombre=Equipo HTTP/1.1" 200 -
{"mensaje": "Hola Equipo, bienvenido a la API de Dulce Hogar"}
luisr@ASUSROMAN:~$
```

Fase 3 — Prueba con cURL hacia /api/saludo: el Reverse Proxy redirige al puerto 9000.



Fase 4 — Sitio publicado por HTTPS en el puerto 8900 sin advertencia de seguridad.

8. Estructura del repositorio

El repositorio `gsw-practico-unidad2-equipo04` está organizado de la siguiente manera:

```
gsw-practico-unidad2-equipo04/
■■■ README.md                        # Integrantes y distribución de roles
■■■ fase1-dimensionamiento.txt       # Fase 1: análisis del caso Dulce Hogar
■■■ fase2-apache.txt                 # Fase 2: instalación y verificación de Apache
■■■ fase3-virtualhost.txt            # Fase 3: VirtualHost y Reverse Proxy
■■■ fase4-https.txt                  # Fase 4: HTTPS local con mkcert
■■■ config/
■   ■■■ dulcehogar.local.conf        # VirtualHost HTTP (referencia)
■   ■■■ dulcehogar.local-ssl.conf    # VirtualHost HTTPS puerto 8900
■   ■■■ ports.conf                   # Puertos de escucha (Listen 8900)
■■■ capturas/                        # Evidencia gráfica de cada fase
    ■■■ fase2-apache-activo.png
    ■■■ fase3-curl-exitoso.png
    ■■■ fase4-https-sin-advertencia.png
```

Nota: en el repositorio los archivos de la carpeta `config/` se encuentran vacíos; su contenido de referencia se documenta íntegramente en la sección de la Fase 4 de este informe.

9. Conclusiones

- El dimensionamiento mostró que el servidor de 2 vCPU y 4 GB RAM es suficiente para el tráfico normal, con un **factor pico de 6** en días festivos; se recomendó escalamiento vertical y, a futuro, migración a la nube por flexibilidad.
- Apache se instaló, habilitó y validó correctamente; la verificación con `apache2ctl configtest` (Syntax OK) resultó clave antes de reiniciar, evitando dejar el sitio caído.
- El VirtualHost con Reverse Proxy permitió exponer la API interna (puerto 9000) a través de la ruta `/api`, ocultando los detalles internos del backend mediante ProxyPass y ProxyPassReverse.
- El uso de **mkcert** habilitó HTTPS local sin advertencias, mostrando las ventajas de una CA de confianza frente a un certificado autofirmado, y dejó claro por qué certbot no aplica para dominios locales.
- La publicación en el puerto **8900** permitió publicar el sitio HTTPS aun con el puerto 80 ocupado por IIS, cumpliendo el requisito de la práctica.
- El trabajo por roles y la documentación colaborativa en Git facilitaron la integración y la trazabilidad de cada fase del laboratorio.